

Table of Contents

ACIT - 4430 Infrastructure Services and Operations Project 1	2
Introduction.....	2
Environment Description and Requirements	2
Infrastructure Provisioning	3
Configuration Management.....	4
Definition of the Workflow	4
Workflow Design	4
Workflow Mapping in Kanban	6
Workflow Test and Internal Validation	6
Experiment Execution and Results	9
Verification of Requirements.....	9
Deployment Clean-up Strategy	10
Compliance with Technical Requirements	11
Reflection	11
Conclusion	12

ACIT - 4430 Infrastructure Services and Operations

Project 1

Introduction

This report documents the design and implementation of a reusable deployment workflow for Alternative 2, which is a storage environment for an archiving department. The objective of the project was not only to deploy the required servers, but to design a structured and repeatable workflow that a junior system administrator can follow in order to deploy the environment consistently and correctly. The focus of the project has therefore been on automation, documentation, and verification.

The deployment environment is based on Infrastructure as Code using Terraform for provisioning virtual machines in OpenStack, and Ansible as the central configuration management system. All servers are provisioned automatically and configured exclusively through Ansible, as required by the assignment.

Environment Description and Requirements

The storage environment consists of four storage servers, one backup server, and one web server. All servers run Ubuntu 24.04 LTS and are deployed in OpenStack using the flavor aem.1c2r.50g, which provides one virtual CPU per instance as required by the assignment.

The four storage servers are intended to run the GlusterFS server package in order to provide distributed storage. The backup server also runs the GlusterFS server package and is intended to support georeplication. The web server functions as a front-end to the storage environment and must run Apache2 with PHP support.

In addition to the service requirements, the assignment specifies identity and access control requirements. On the storage servers, two users named kirk and rich must be created. Both users must be granted sudo privileges and must be members of a group called storageadmins. The group storageadmins must exist on both the storage servers and the backup server. On the web server, a user named janet must be created. Since the assignment does not specify that janet requires sudo privileges, no sudo access has been granted to this user. The overall architecture of the deployment environment is illustrated in Figure 1 below.

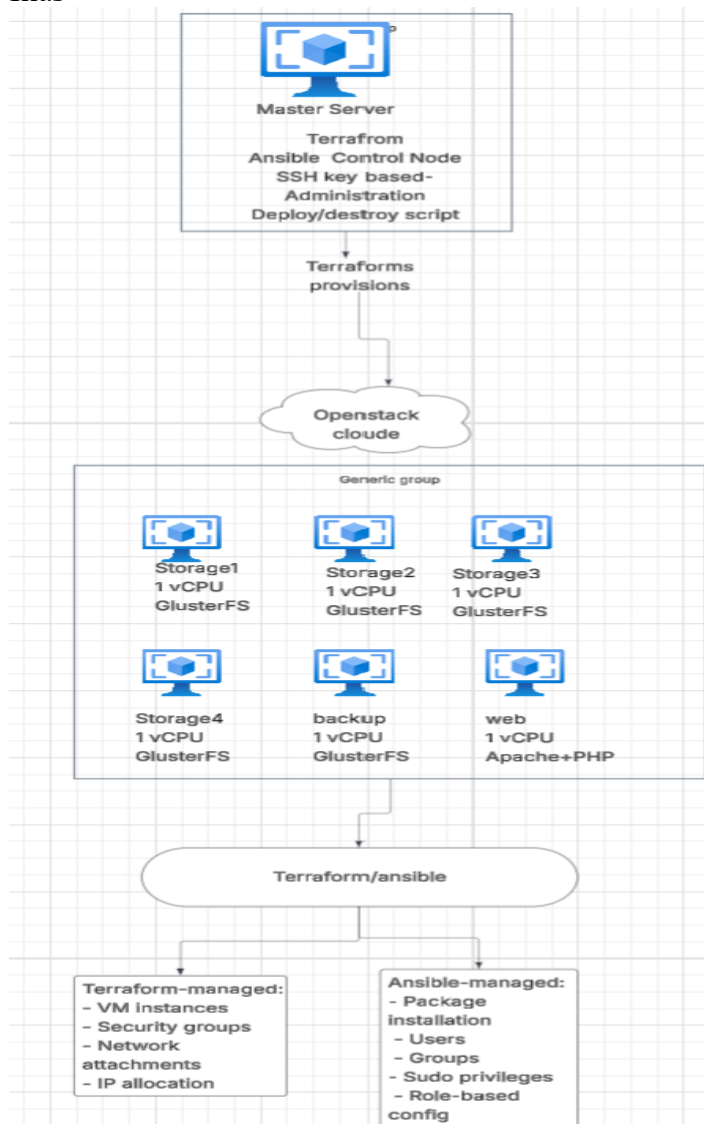


Figure 1: High-level architecture of the Alternative 2 storage deployment environment.

The architecture diagram in Figure 1 illustrates the overall structure of the deployment environment. The master server functions both as the Infrastructure as Code execution node using Terraform and as the configuration management control node using Ansible. Terraform is used to provision the virtual machines in OpenStack from the master server. Once the instances are created, Ansible connects to them via key-based SSH in order to install the required packages and configure users and groups. The servers are configured so that they can only be accessed from the master server through SSH. The four storage nodes form a distributed storage layer based on GlusterFS, the backup server supports replication, and the web server provides a front-end interface using Apache2 and PHP.

Infrastructure Provisioning

The infrastructure is provisioned using Terraform from the master server in the directory `project1/terraform`. Separate Terraform configuration files are used to define the storage servers, backup server, and web server. Security groups are defined for each server type in order to allow necessary network access and SSH access is enabled for all servers.

Ilias

The four storage servers are created dynamically using a local set definition combined with a “for_each loop”, which ensures that the configuration is reusable and scalable. Each storage node is provisioned with the same base image, flavor, key pair, and network configuration. The backup and web servers are provisioned in a similar manner with their respective security group rules.

All servers use key based authentication via the Master key pair, ensuring that they can be accessed securely from the master server without password based login. Terraform outputs are used to retrieve the IP addresses of the deployed instances. This provisioning approach ensures that the infrastructure can be recreated in a consistent and repeatable way, illustrated in figure 6 and 7.

Configuration Management

After the virtual machines are provisioned, all configuration is performed using Ansible. The Ansible inventory is located in /etc/ansible/hosts on the master server. The relevant host groups used in this project are storage, backup, and web, which correspond directly to the roles defined in the main playbook site.yml.

The playbook applies three roles: one for storage servers, one for the backup server, and one for the web server. This separation of roles ensures clear responsibility and modular configuration.

On the storage servers, the Ansible role installs the glusterfs-server package, creates the group storageadmins, creates the users kirk and rich, adds both users to the storageadmins group, and grants them sudo privileges. On the backup server, the glusterfs-server package is installed and the storageadmins group is created. The users kirk and rich are added to this group as required. On the web server, the apache2 and php packages are installed and the user janet is created.

All software installation and user configuration tasks have been performed exclusively through Ansible, in accordance with the requirement that all deployed servers must be managed by a central configuration management system.

Definition of the Workflow

In this project, the workflow refers to the sequence of operational steps required to deploy the requested environment, not the process of building the underlying infrastructure such as the master server or installing Terraform and Ansible. The workflow assumes that the deployment platform is already in place. Its purpose is to describe how a junior system administrator can receive a deployment request and execute the necessary steps to provision, configure, and verify the storage environment.

Workflow Design

The workflow was designed to be simple and reproducible. It is structured so that a junior system administrator can deploy the full environment by following documented steps without modifying infrastructure code.

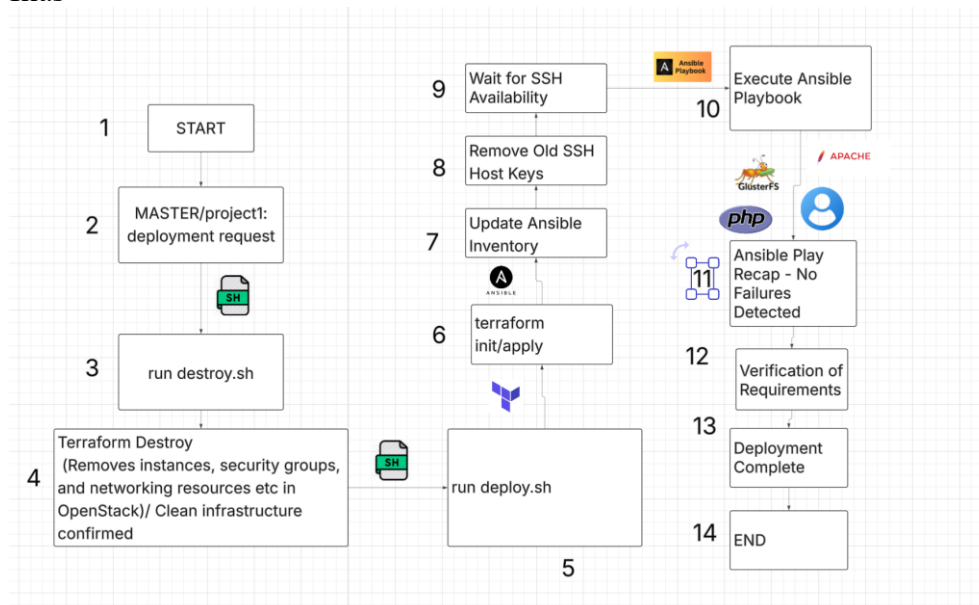


figure 2 Operational Deployment Workflow

As shown in Figure 2, the workflow moves step by step from **1** to **14**. In steps **1-2**, a deployment request is handled on the master node where the project scripts are located. In steps **3-4**, the destroy script is executed, and Terraform removes all existing instances, security groups, and networking resources in OpenStack to make sure the environment starts from a clean state. In steps **5-6**, the deploy script is run, and Terraform initializes and provisions the required infrastructure, including the storage nodes, backup node, and web server. Steps **7-9** prepare the servers for configuration by updating the Ansible inventory with new IP addresses, removing old SSH host keys, and waiting until all machines are reachable via SSH. In step **10**, the Ansible playbook is executed to configure the servers, install required services such as GlusterFS, Apache, and PHP, and create the necessary users and groups. Step **11** confirms that the playbook completed successfully with no failures, and step **12** verifies that all assignment requirements are met. Finally, in steps **13-14**, the deployment is confirmed as complete and the workflow ends.

The workflow clearly separates infrastructure preparation from configuration management. Terraform is responsible for creating the virtual machines and associated infrastructure, while Ansible ensures consistent configuration of all servers based on their group membership in the inventory file.

The final stage of the workflow consists of verifying that all requirements have been fulfilled. Verification is performed by logging into the relevant servers and checking that required packages are installed, that the storageadmins group exists, that the specified users have been created, and that sudo privileges have been correctly assigned where required.

The workflow has been mapped into Trello using WIP cards to represent deployment tasks, identity configuration, privilege configuration, and verification steps. This allows tracking of time, progress, and improvements over multiple deployment runs

Workflow Mapping in Kanban

The workflow was mapped into a Kanban board in Trello to visualize development tasks, internal workflow tests, and external experiment execution. During development, tasks were placed in the “inbox” column and then “pågå” and finally to “Done” once completed. For internal validation, three separate cards were created under “workflow tests internal validation”, representing the three full destroy and deploy verify cycles. During the experiment phase, tasks were first placed in “Eksperiment testing on group” and moved to “Done Experiment section only” after successful verification a, as illustrated in figure 3.

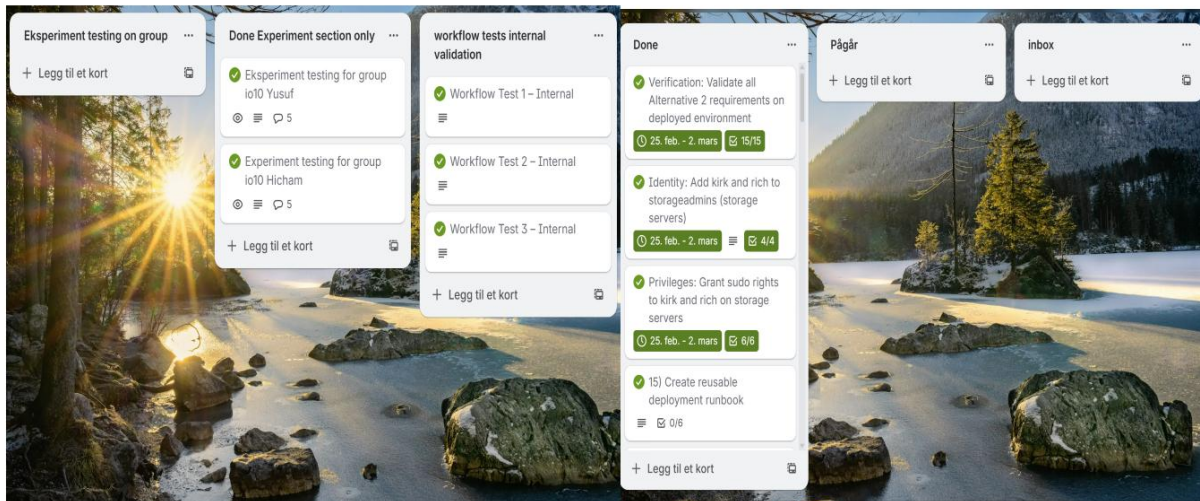


Figure 3: Complete Kanban board showing development tracking, internal workflow validation, and experiment execution.

Mapping the workflow visually also helped clarify the difference between setup and configuration, which is central to the purpose of this assignment. By organizing the workflow into distinct operational phases, the deployment process becomes easier for a junior system administrator to follow and repeat.

Workflow Test and Internal Validation

In order to evaluate the stability and reproducibility of the defined workflow, the deployment process was executed three complete times from start to finish. Each test consisted of destroying the existing environment and redeploying it using the automated script that integrates Terraform setup and Ansible configuration. The objective was to verify that the workflow could consistently recreate the environment without manual intervention.

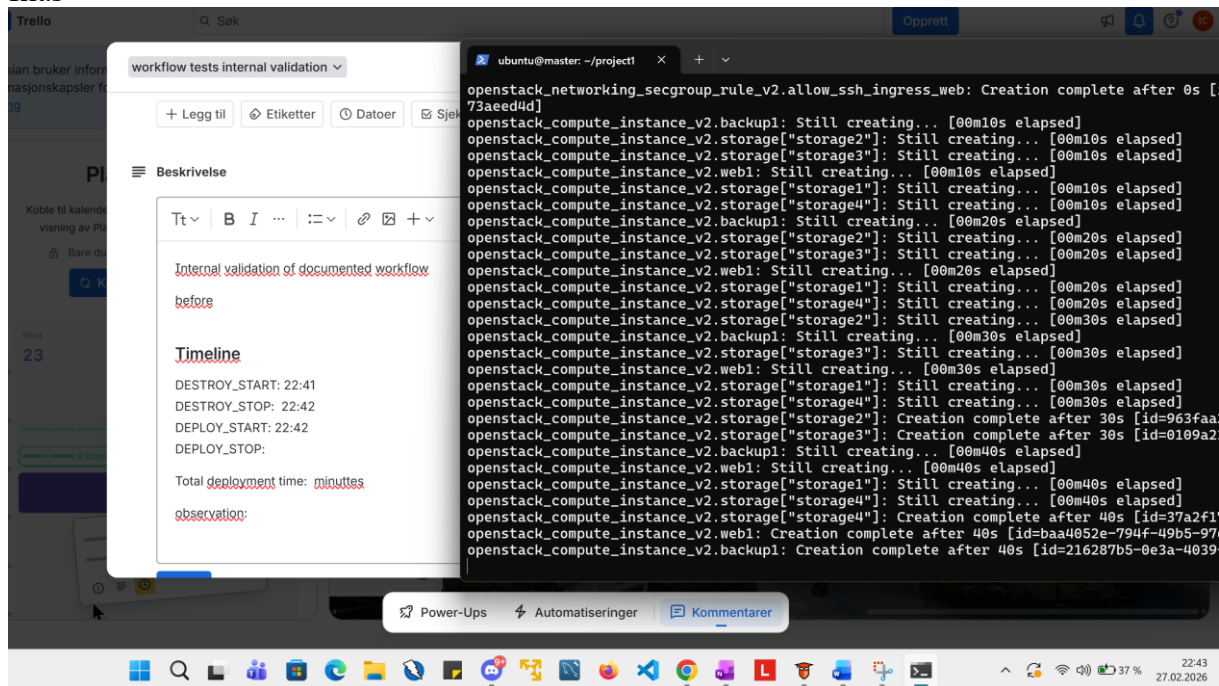


Figure 4 shows the Terraform setup phase during one of the internal workflow validation tests.

During the first internal test, a minor issue was discovered. The backup role did not install the glusterfs-server package as required by the assignment. This was identified during the verification phase using remote package inspection commands. The issue was traced back to the Ansible backup role configuration and corrected by adding the required package installation task. After the fix was implemented, the workflow was re-executed and successfully completed.

The second and third tests were executed without any modifications to the codebase. Both destroy and deploy operations completed successfully, and all required users, groups, and services were verified to be correctly installed on their respective servers. The successful completion of the Ansible deploy configuration phase during one of the workflow validation tests is shown in Figure 5.

```
TASK [web : Update apt cache] *****
changed: [web1]

TASK [web : Install Apache2] *****
changed: [web1]

TASK [web : Install PHP] *****
changed: [web1]

TASK [web : Ensure Apache is running] *****
ok: [web1]

TASK [web : Create user janet] *****
changed: [web1]

PLAY RECAP *****
backup1      : ok=5    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
storage1     : ok=8    changed=7    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
storage2     : ok=8    changed=7    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
storage3     : ok=8    changed=7    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
storage4     : ok=8    changed=7    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
web1         : ok=6    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

Deploy ferdig
ubuntu@master:~/project1$ |
```

Figure 5: Ansible play recap showing successful configuration of all storage, backup, and web nodes with no failures or unreachable hosts.

IP addresses were regenerated between deployments as expected, confirming that the environment was rebuilt from a clean state each time.

```
ubuntu@master:~/project1$ ls
ansible scripts terraform workflow
ubuntu@master:~/project1$ sed -n '/^\[storage\]/,/^\$/p; /^\[backup\]/,/^\$/p; /^\[web\]/,/^\$/p' /etc/ansible/hosts
[storage]
storage1 ansible_host=10.196.243.221
storage2 ansible_host=10.196.245.42
storage3 ansible_host=10.196.241.5
storage4 ansible_host=10.196.240.135

[backup]
backup1 ansible_host=10.196.241.194

[web]
web1 ansible_host=10.196.244.214

ubuntu@master:~/project1$ |
```

```
ubuntu@master:~/project1$ sed -n '/^\[storage\]/,/^\$/p; /^\[backup\]/,/^\$/p; /^\[web\]/,/^\$/p' /etc/ansible/hosts
[storage]
storage1 ansible_host=10.196.245.19
storage2 ansible_host=10.196.242.151
storage3 ansible_host=10.196.242.27
storage4 ansible_host=10.196.244.19

[backup]
backup1 ansible_host=10.196.243.29

[web]
web1 ansible_host=10.196.245.129

ubuntu@master:~/project1$ |
```

Figure 6-7 illustrates the change in IP addresses between two consecutive deployments, demonstrating that new instances were provisioned after each destroy operation.

The measured total workflow durations were as follows:

- **Test 1: 7 minutes**
- **Test 2: 6 minutes**
- **Test 3: 5 minutes**

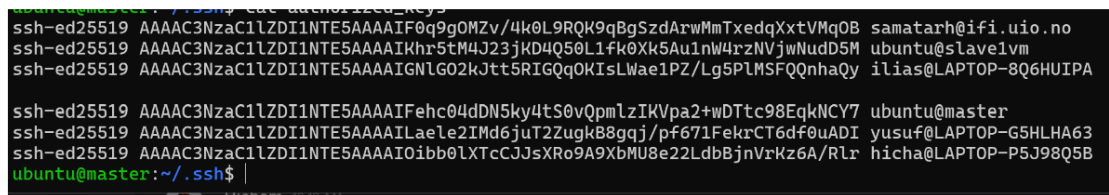
This shows a decreasing deployment time as familiarity with the workflow increased and no adjustments actions were required in later runs. The average total workflow duration across the three tests was approximately 6 minutes.

The internal testing demonstrates that the workflow is stable, automated, and reproducible. After correcting the initial configuration error, all subsequent deployments were completed successfully without manual adjustments, which shows a high level of technical reliability.

Experiment Execution and Results

To evaluate whether the workflow could be followed by someone outside our group, we conducted an experiment where two students were invited to execute the deployment process using only our documented instructions. The goal was to test if the workflow was clear enough to be understood and executed without technical assistance.

Before the experiment started, the participants were granted access to the master server using key based SSH authentication. They provided their public SSH keys, which were added to the master server's `authorized_keys` file. This ensured secure access while maintaining control over who could connect, as illustrated in figure 8



```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIF0q9gOMZv/4k0L9RQK9qBgSzdArwMmTxedqXxtVMqOB samatarh@ifi.uio.no
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKhr5tM4J23jKD4Q50L1fk0Xk5Au1nw4rzNVjwNudD5M ubuntu@slave1vm
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGNlG02kJtt5RIGQq0KIsLWae1PZ/Lg5PLMSFQqnhaQy ilias@LAPTOP-8Q6HUIPA

ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFehc04dDN5ky4tS0vQpmlzIKVpa2+wDTtc98EqkNCY7 ubuntu@master
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAILaeLe2IMd6juT2Zugk8Bgqj/pf671FekrCT6df0uADI yusuf@LAPTOP-G5HLHA63
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIoibb0LXTcCJJsXR09A9XBMU8e22LdbBjnVrKz6A/Rlr hicha@LAPTOP-P5J98Q5B
ubuntu@master: ~/.ssh$
```

Figure 8 shows the SSH public keys added to the master server during the experiment.

During the execution, we maintained communication while they were performing the steps. As they ran the workflow, I manually recorded the timestamps for when the destroy process started, when the destroy process finished, and when the deployment started. This allowed us to track the duration of each phase of the workflow.

Once the deployment finished successfully on their side, they were instructed to comment “verified = [time]” in the Trello card. This served as confirmation that the environment was fully deployed and functioning correctly from their perspective. The Trello activity log therefore acted as both documentation and proof that the workflow had been executed successfully.

Both participants were able to complete the process without requiring technical intervention. The destroy and deployment scripts executed successfully, the infrastructure was recreated from a clean state, and all required services were installed and verified as expected. The experiment confirmed that the documented workflow is clear, reproducible, and practically applicable for execution by an external user following the provided instructions. Furthermore, the experiment was conducted in both directions, meaning that our group also executed the workflow on the other group's (IO10) deployment environment. This cross validation strengthened the credibility and reliability of the experimental results.

Verification of Requirements

Verification of the deployment is performed manually using non destructive commands. On the storage and backup servers, the installation of the glusterfs server package is confirmed

using package listing commands. The existence of the storageadmins group is verified, and membership of kirk and rich is confirmed using identity commands. On the storage servers, sudo privileges for kirk and rich are verified using sudo inspection commands.

On the web server, the installation of apache2 and php is verified, and the existence of the janet user is confirmed. Since no additional configuration of GlusterFS or Apache was required by the assignment, installation of the packages is considered sufficient.

```

ubuntu@kali:~$ ansible-all -m ping
[WARNING]: Host 'storage1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter
could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html
for more information.
storage1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}

[WARNING]: Host 'storage2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter
could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html
for more information.
storage2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}

[WARNING]: Host 'backup1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter
could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html
for more information.
backup1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}

[WARNING]: Host 'storage3' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter
could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html
for more information.
storage3 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}

[WARNING]: Host 'storage4' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter
could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html
for more information.
storage4 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}

[WARNING]: Host 'storage5' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter
could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html
for more information.
storage5 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}

[WARNING]: Host 'web1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter
could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html
for more information.
web1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}

ubuntu@kali:~$

```

The screenshot above demonstrates successful installation of required packages, correct user and group configuration, and verified connectivity across all managed nodes. No errors were detected during verification.

In order to ensure that the environment can be redeployed multiple times, a clean-up strategy has been defined as part of the workflow. Terraform is used not only for provisioning infrastructure but also for removing it. By executing terraform destroy from the master server, all virtual machines and associated resources created during the deployment are removed in a controlled and automated manner.

This approach ensures that the environment can be recreated from a clean state before running additional workflow tests. Including removal as part of the workflow reduces configuration drift and strengthens the reproducibility of the deployment process.

```

openstack_networking_secgroup_v2.web_security_group: Destroying... [id=50940f85-345e-4451-9102-16837b8c1fe4]
openstack_networking_secgroup_v2.storage_security_group: Destroying... [id=2993a361-83ec-4993-bf69-1e25ec904543]
openstack_networking_secgroup_v2.backup_security_group: Destruction complete after 8s
openstack_networking_secgroup_v2.web_security_group: Destruction complete after 8s
openstack_networking_secgroup_v2.storage_security_group: Destruction complete after 8s

Warning: Argument is deprecated

  with provider["registry.terraform.io/terraform-provider-openstack/openstack"],
  on versions.tf line 10, in provider "openstack":
  10: provider "openstack" {

Users not using loadbalancer resources can ignore this message. Support for neutron-lbaas will be removed on next major release. Octavia will be
the only supported method for loadbalancer resources. Users using octavia will have to remove 'use_octavia' option from the provider
configuration block. Users using neutron-lbaas will have to migrate/upgrade to octavia.

(and 2 more similar warnings elsewhere)

Destroy complete! Resources: 13 destroyed.
Destroy ferdiq
ubuntu@master:~/project1$ |

```

Figure 10: Terraform destroy execution confirming complete infrastructure removal.

The screenshot confirms that all infrastructure resources were successfully removed before initiating a new deployment cycle.

Compliance with Technical Requirements

All deployed servers are managed exclusively through Ansible, which acts as the central configuration management system. No manual package installation or user configuration has been performed outside of Ansible.

All servers allow key based SSH login from the master server using the configured key pair. Password-based authentication has not been used during deployment.

```

ubuntu@master:~$ ssh -o PasswordAuthentication=no \
-o PubkeyAuthentication=yes \
ubuntu@10.196.242.113 'echo SSH_OK_$(hostname) '
SSH_OK_storage1
ubuntu@master:~$ ssh -o PasswordAuthentication=no -o PubkeyAuthentication=yes ubuntu@10.196.245.60 'echo SSH_OK_
$(hostname) '
SSH_OK_storage2
ubuntu@master:~$ ssh -o PasswordAuthentication=no -o PubkeyAuthentication=yes ubuntu@10.196.241.250 'echo SSH_OK
_$(hostname) '
SSH_OK_storage3
ubuntu@master:~$ ssh -o PasswordAuthentication=no -o PubkeyAuthentication=yes ubuntu@10.196.241.121 'echo SSH_OK
_$(hostname) '
SSH_OK_storage4
ubuntu@master:~$ |

```

Figure 11: Verification of key-based SSH access from the master server to all deployed nodes.

The deployment process is highly automated. Infrastructure provisioning is handled entirely by Terraform, while configuration management is executed through Ansible roles. Manual interaction is limited to initiating the workflow and performing verification.

Reflection

The combination of Terraform and Ansible provides a clear separation between infrastructure provisioning and server configuration. Terraform ensures that the infrastructure can be recreated consistently, while Ansible guarantees uniform configuration across all nodes.

One strength of the workflow is its modular structure. The use of separate roles for storage, backup, and web servers improves readability and maintainability. Additionally, the use of dynamic provisioning for storage nodes demonstrates scalability and reusability.

Possible improvements include introducing automated verification tasks in Ansible to further reduce manual checks. The workflow could also be extended with improved error handling documentation for junior administrators.

Conclusion

This project demonstrates the design and implementation of a repeatable deployment workflow for a storage environment in OpenStack. By combining Infrastructure as Code with centralized configuration management, the environment can be provisioned and configured in a consistent and automated manner. The workflow is structured so that a junior system administrator can follow it step by step and deploy the required environment while meeting all technical and identity related requirements defined in the assignment.